Arquitectura en capas y patrones

1. Modelo de arquitectura

Flujo de la peticion: urls.py -> views.py -> services.py -> models.py. El endpoint usa JWT mediante IsAuthenticated.

Patron Strategy

2. Capa de servicios

Las estrategias intercambian la regla de descuento segun la categoria del producto.

products/services.py

```python
class EstrategiaDescuento:
    def calcular(self, precio):
        raise NotImplementedError

class DescuentoElectronica(EstrategiaDescuento):
    def calcular(self, precio):
        return precio * 0.90

class DescuentoRopa(EstrategiaDescuento):
    def calcular(self, precio):
        return precio * 0.80

def obtener_estrategia(categoria_nombre):
    estrategias = {'Electronica': DescuentoElectronica(), 'Ropa': DescuentoRopa()}
    return estrategias.get(categoria_nombre, SinDescuento())
```

Patron Factory Method

3. Creacion con categoria por defecto

El metodo centraliza la creacion y asigna Sin categoria cuando el cliente no proporciona una.

products/models.py

```python
@classmethod
def crear_con_categoria_default(cls, name, description, price):
    categoria_default, _ = Category.objects.get_or_create(
        name='Sin categoria')
    return cls.objects.create(name=name, description=description,
        price=price, category=categoria_default)
```

Patron Facade

4. Endpoint protegido

Una sola llamada oculta la busqueda del producto, la seleccion de estrategia y el calculo del precio.

products/views.py y products/urls.py

```python
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def precio_final(request, pk):
    producto = get_object_or_404(Product, pk=pk)
    estrategia = obtener_estrategia(producto.category.name)
    precio = estrategia.calcular(float(producto.price))
    return Response({'producto': producto.name,
        'precio_original': producto.price,
        'precio_final': round(precio, 2)})

path('products/<int:pk>/precio-final/', precio_final)
```

Respuestas autenticadas

```
{"producto":"Laptop", "precio_final":13500.0}
{"producto":"Sudadera", "precio_final":800.0}
```

Pruebas unitarias

5. Resultado de la ejecucion

Las pruebas comprueban los dos descuentos y que una peticion sin autenticacion reciba 401.

products/tests.py

```
def test_precio_final_autenticado(self):
    self.assertEqual(response_electronica.status_code, 200)
    self.assertEqual(response_ropa.status_code, 200)
    self.assertEqual(response_electronica.data['precio_final'], 13500.0)
    self.assertEqual(response_ropa.data['precio_final'], 800.0)

def test_precio_final_sin_autenticar(self):
    self.client.force_authenticate(user=None)
    self.assertEqual(response.status_code, 401)
```

PowerShell - manage.py test

```
Found 8 test(s).
........
----------------------------------------------------------------------
Ran 8 tests in 3.281s

OK
```

Tabla de patrones

6. Evidencia y explicacion

Factory Method: creacional, models.py, encapsula la instanciacion. Strategy: comportamiento, services.py, define algoritmos intercambiables. Facade: estructural, views.py, ofrece una interfaz unica.